Source: https://cam.feedzai.com/cs-hsbc/3.0.2/docs/operations-guide/monitoring/top-metrics-pulse

On this page

Top Metrics to Monitor in Pulse

In this section you will find which metrics are important to monitor in **Pulse**.

info

Note that the following list is simply a reference to the most important metrics, and should not be considered as the single source of monitoring the healthiness of the system.

Please refer to Understand Available Metrics for more information.

note

The following Pulse standard application tags are available in every metric that references `Standard Pulse application tags`:

- `metricType`
- `appType`
- `appSlug`

warning

Some metrics will only be available in the metrics endpoint if the value is greater than zero, otherwise it might be expected for them to not appear at all.

Workflowâ□□

Workflow Throughputâ□□

Metric	Description	Tags	Type
`pulse.workflow.global.event.stats`	Measures the global throughput of the selected workflow belonging to the specified appSlug. It displays the throughput per message type (normal, recovery and replica).	• `workflowDesc` • `workflowId` • `messageType` • Standard Pulse application tags	Counter

Because this metric is captured for each type of message, the messageType tag allows you to distinguish between:

- **NORMAL** - Used for news messages that arrive at workflow.
- **RECOVERY** - Used for messages that are received when the system is doing recovery during a publish operation.
- **REPLICA** - Used to propagate the result of a NORMAL message, to all other Pulse instances.

What to monitorâ□□

This metric is useful to ensure that the rate at which events are being processed is as expected. If you are injecting events at 100 events per second into the workflow, this metric should report a similar value. If the rate being reported is consistently below the expected number, this might indicate that events are not being processed fast enough. This can indicate that there is a problem somewhere in the system.

Workflow Latenciesâ□□

Metric	Description	Tags	Type
`pulse.workflow.global.event.stats`	Measures the global latencies per percentile (50, 75, 95, 99 and 99.9), of the selected workflow belonging to the specified appSlug.	• `workflowDesc` • `workflowId` • `messageType` • Standard Pulse application tags	Timer

Because this metric is captured for each type of message, the `messageType` tag allows us to distinguish between **NORMAL**, **RECOVERY** and **REPLICA** messages.

What to monitorâ□□

This metric is useful together with the previous throughput metric. As it reports the latency of the events being processed, if the throughput starts to decrease, we can use this metric as a complement to understand what can be contributing to that -- for example, the system might be reaching memory exhaustion (see JVM metrics), or itâ□□s taking to long to fetch historical profiles from storage (see Profile Fetch Latencies metrics).

Workflow Errorsâ□□

Metric	Description	Tags	Type
`pulse.workflow.eventErrors.count`	Measures the global workflow errors registered on the selected pulse instance, appSlug and appSlug workflow.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Counter

What to monitorâ□□

This metric provides a simple way to understand if there are any errors during event processing in a workflow.

When there are no errors, the value of this counter is zero. If this metric reports a different value, then you must look into the log files for additional error details.

Workflow Validationâ□□

Metric	Description	Tags	Type
`pulse.workflow.validation.concurrent`	Measures the number of concurrent workflow validations.	• `metricType` • `appType`	Gauge
`pulse.workflow.validation.e2e`	Measures end-to-end workflow validation time: since the request is received in Pulse Server until the answer is returned to the user.	• `workflowDesc` • `workflowId` • `metricType` • `appType`	Gauge
`pulse.workflow.validation.remote`	Measures time spent doing a remote call to the app engine for workflow validation.	• `workflowDesc` • `workflowId`	Gauge

Metric	Description	Tags	Type
		• `metricType` • `appType`	

What to monitorâ□□

These metrics refer to the work validations which are issued whenever a user opens/edits a workflow.

These validations may take from seconds to minutes depending on the complexity of a workflow.

Eventsâ□□

Event Pending Persistence to Event Storageâ□□

Metric	Description	Tags	Type
`pulse.workflow.eventstorage.eventsQueuedSize`	Measures the number of events that are pending to be persisted to the Event Storage (on DB).	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Counter

What to monitorâ□□

Events are persisted in batches and at periodic intervals. Both the size of the batch and interval at which the batch persistence is triggered are configurable in the Pulse Administration page.

The Pulse properties are:

- `services.modules.rte.workflowmanager.eventstorage.databasebatch.size`
- `services.modules.rte.workflowmanager.eventstorage.databasebatch.timeout`

As such, the number reported by this metric shouldnâ□□t greatly exceed the size configured for the batch (which defaults to 10000).

Custom Alert Storage Service Additional Fieldsâ□□

Metric	Description	Tags	Type
`hsbc.workflow.ass.missingExpectedFields.count`	Number of additional fields configured to enrich events that are missing from the workflow schema.	N/A	Counter

Workflow Eventsâ□□

Metric	Description	Tags	Type
`workflow.events`	Measures the events being processed by each workflow belonging to the specified appSlug.	Check detailed description below	Meter

Unlike the Workflow Throughput metrics, this metric only takes normal messages into account.

The following tags are available to further breakdown the events being processed:

- `fdz_channel` : describes the channel of the event.
- `event_type` : describes the event type of the event.
- `decision` : describes the workflow decision outcome, for the event.
- `fraud` : describes the workflow fraud outcome, for the event.
- `Alert` : describes whether the event resulted in an alert.
- `appSlug` : describes the appSlug.

- `tenant` : describes the tenant.
- `workflowId` : describes the workflowId associated with the event.

What to monitorâ□□

This metric provides a comprehensive breakdown of the events being processed. Its uses include, but are no limited to:

• roughly counting the number of events flowing in the system (possibly by channel or even event type)
• roughly monitoring the decline rate of the system (possibly by channel or even event type)
• roughly monitoring the alert rate of the system (possibly by channel or even event type).

Events Dumped to Diskâ□□

Metric	Description	Tags	Type
`pulse.dump.storage.dump.entries.retry.counter`	The total number of events that are currently stored in the disk to be retried by the auto recovery job.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Counter
`pulse.dump.storage.dump.entries.failed.counter`	The total number of events that are currently stored in the disk but that already reached the maximum amounts of retry, therefore are no longer reprocessed.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Counter

Measures the number of events dumped to disk per host, appSlug, and workflow element. When events reach the end of the workflow they are persisted in the Event Storage (Database). If the Event Storage isn't available (e.g. The Database is offline, there's a network error, or some other unexpected error) the events are dumped to disk and are reprocessed later on by the Event Storage Dump component.

What to monitorâ□□

Events are not expected to be dumped to disk. Therefore, this metric should report zero. Otherwise, this indicates that some events are failing to be persisted to the DB, which suggests an issue in the system.

Events spilled to Diskâ□□

Metric	Description	Tags	Type
`pulse.eventStorage.spillToDisk.count`	The number of events that failed persistence to event storage and that were dumped into local disk.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Counter
`pulse.eventStorage.spillToDisk.pendingQueue`	The number of events that failed persistence to event storage and that are pending in-memory to be dumped into local disk.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Gauge

Measures the number of events spilled to disk per host, appSlug and workflow element. Events are spilled to disk when, after being processed by the workflow, they fail to be persisted to the Event Storage (on the DB). This might happen because the DB is down, thereâ□□s a network error, a timeout occurred, or some other unexpected error.

What to monitorâ□□

Events are not expected to be dumped to disk. Therefore, this metric should report zero. Otherwise, this indicates that some events are failing to be persisted to the DB, which suggests an issue in the system.

Auto Recovery Jobâ

Metric	Description	Tags	Type
`pulse.dump.storage.retry.queue.size`	Number of in-memory entries waiting to be written to the retry folder.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Gauge
`pulse.dump.storage.failed.queue.size`	Number of in-memory entries waiting to be written to the failed folder.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Gauge
`pulse.dump.storage.recovery.job`	Whether the auto recovery job is running or not.	• `workflowDesc` • `workflowId` • Standard Pulse application tags	Gauge

These sets of metrics capture the current status of the auto recovery job used for persisting Events Dumped to Disk.

These metrics are available under the `pulse.dump` prefix. To capture a specific workflow, use the `workflowDesc` tag. As an example, to capture auto recovery job metrics for the `Payments` workflow, look for `workflowDesc=Payments`.

What to monitorâ

These metrics are important to know if the recovery job may have issues processing the spilled to disk events and for how long it is running. For instance, if the recovery job runs for several hours it may be due to database issues or because the Recovery Job is taking too much time to process events.

Jobsâ

Scheduled Jobsâ

These sets of metrics capture the current status of all scheduled jobs running in the system.

Metric	Description	Tags	Type
`pulse.job.executions.scheduled`	Number of jobs that are scheduled.	• `jobType` • Standard Pulse application tags	Counter
`pulse.job.executions.waiting`	Number of jobs that are waiting.	• `jobType` • Standard Pulse application tags	Counter
`pulse.job.executions.executing`	Number of jobs that are executing.	• `jobType` • Standard Pulse application tags	Counter
`pulse.job.executions.failed`	Number of jobs that failed.	• `jobType` • Standard Pulse application tags	Counter
`pulse.job.executions.crashed`	Number of jobs that crashed.	• `jobType`	Counter

Metric	Description	Tags	Type
		• Standard Pulse application tags	
`pulse.job.executions.aborted`	Number of jobs that were aborted.	• `jobType` • Standard Pulse application tags	Counter

What to monitorâ□□

These metrics are especially important to detect jobs that have either failed, crashed or been aborted, as this is a sign that something went wrong -- in which case the log files will provide more detailed information.

Profile Fetches Latenciesâ□□

Metric	Description	Tags	Type
`pulse.workflow.profileFetches.statistics`	Provides statistics on the latency of fetching these profiles from the external storage.	• `elementId` • `workflowDesc` • `workflowId` • `elementDesc` • Standard Pulse application tags	Timer

What to monitorâ□□

This depends on the use case. Nevertheless, make sure you understand what are the expected latencies, and look out for any continuous deviation from those values. If the latency starts to increase, there could be an underlying problem with the profile store in Cassandra. Look in the log files for more details.

Pulse Application Statesâ□□

Pulse Statusâ□□

As an application server, Pulse is responsible for managing the lifecycle of the applications it serves. This lifecycle varies with the state at which the application is. For example, if an application is healthy and running as expected, it is said to be in state `STARTED`. However, if someone publishes a change, it transitions to state `APPLYING_CHANGES`.

How to monitorâ□□

There are 10 possible application states, and these are being reported as an integer between 1 and 10. The states and their corresponding value are:

- **STOPPING(1)**: Application is stopping. For each AppEngine, this means that it is shutting down the JVM.
- **STOPPED(2)**: Application has stopped. Therefore, it should not have any JVM process.
- **BOOTSTRAPPING(3)**: Application is bootstrapping, i.e. services are being instantiated. This means that each AppEngine in this state is starting the JVM and creating the services, which haven't started yet.
- **BOOTSTRAPPED(4)**: Application has been bootstrapped. This means that for each AppEngine in this state, the respective JVM is up and running and all services have been created (but not started).
- **STARTING(5)**: Application is starting. This means that each AppEngine is starting its services since they were only instantiated in the previous state.
- **STARTED(6)**: Application has started. This means that, for each AppEngines, the services have started and the definitions that were meant to be published have been loaded. Both the AppEngine and app as a whole transition to this state from `APPLYING_CHANGES`.
- **CRASHED(7)**: Application has crashed. This means that it does not have any AppEngine alive. Each AppEngine never assigns itself this state. This is not possible since its JVMs are off.

- **APPLYING_CHANGES(8)**: Application is applying changes. This includes loading (or unloading) the definitions to be published. The AppEngine may have transitioned to this state from `STARTED` (in case it's re-publishing an already published app) or from `STARTING` (in case it's publishing an app that was previously in the `BOOTSTRAPPED` state). In either case, and if the AppEngine is working properly, it will transition to the `STARTED` state.
- **DEGRADED(9)**: Application is in degraded state, i.e., there may be partitions with no replicas online and/or there are replicas running with distinct versions. This means that some AppEngines lack JVMs, and therefore they can't assign themselves this state. The healthy AppEngines do not transition to this state either.
- **SELF_HEALING(10)**: This is the cluster's overall state and it means that the application is being self-healed by a watchdog correction. The individual AppEngines do not transition to this state. This is an overall app state only.

All AppEngines' states are available under the below metrics. To capture a specific AppEngine, use the `appSlug` tag.

Metric	Description	Tags	Type
`pulse.app.state`	The state of the Pulse Application.	• Standard Pulse application tags	Gauge
`pulse.server.state`	The state of the Pulse Server.	• Standard Pulse application tags	Gauge

Note that the state of each application is shown via the Pulse interface.

What to monitor in Pulse Application Statesâ

It is normal for applications to switch between states as teams promote new changes. However, it is not normal to see the application stop, crash, or degrade. Tracking the state of the application is crucial to identify silent problems that may require manual intervention, such as repairing a cluster that has replicas running with different versions of Risk Strategy.

App Lifecycle Operationsâ

These metrics allow for monitoring app engine bootstrap and upstart operations.

Metric	Description	Tags	Type
`pulse.app.publish.count`	The number of times the AppEngine got a publish request since startup.	• Standard Pulse application tags	Counter
`pulse.app.publish.skipRecovery`	Tracks the number of times recovery operations have been skipped.	• Standard Pulse application tags	Counter
`pulse.app.publish.applyingChanges.state`	Reports the fine-grained states during the `APPLYING_CHANGES` phase. For more details, please consult Fine-grained Application States.	• Standard Pulse application tags	Gauge
`pulse.app.publish.bootstrap.success`	Measures the full duration from app process start until the bootstrap work finishes successfully.	• Standard Pulse application tags	Timer
`pulse.app.publish.upstart.success`	Measures the end-to-end upstart time in success cases (i.e. from when the upstart message is received until it finishes successfully).	• Standard Pulse application tags	Timer
`pulse.app.publish.upstart.error`	Measures the end-to-end upstart time in error cases (i.e. from when the upstart message is received until it finishes with an error).	• Standard Pulse application tags	Timer
`pulse.app.publish.upstartBeforeWorkflows.success`	Measures the time from upstart message reception until workflows start being processed (for validation, recovery, etc.).	• Standard Pulse application tags	Timer
			Timer

Metric	Description	Tags	Type
`pulse.app.publish.loadAndValidateWorkflows.success`	Measures the workflow loading and validation time in success cases (i.e. when all workflows loading/validation succeed).	• Standard Pulse application tags	
`pulse.app.publish.loadAndValidateWorkflows.error`	Measures the workflow loading and validation time in error cases (i.e. when at least one workflow loading/validation fails).	• Standard Pulse application tags	Timer
`pulse.app.publish.recovery.success`	Measures the workflow recovery time in success cases (i.e. when all workflows recover successfully).	• Standard Pulse application tags	Timer
`pulse.app.publish.recovery.error`	Measures the workflow recovery time in error cases (i.e. when at least one workflow fails recovery).	• Standard Pulse application tags	Timer
`pulse.app.publish.startWorkflows.success`	Measures the workflow start time in success cases (i.e. when all workflows start successfully).	• Standard Pulse application tags	Timer
`pulse.app.publish.startWorkflows.error`	Measures the workflow start time in error cases (i.e. when at least one workflow fails to start).	• Standard Pulse application tags	Timer

Fine-grained Application Statesâ

There are 10 possible fine-grained application states during the `APPLYING_CHANGES` state, reported as an integer between 0 and 6. The states and their corresponding value are:

- **IDLE(0)**: State reported when `pulse.app.state` is not `APPLYING_CHANGES`.
- **UNLOADING(1)**: Application is unloading services/workflows.
- **LOADING_BEFORE_WORKFLOWS(2)**: Application is is loading services before reaching workflow code (validation/recovery/start).
- **LOADING_AND_VALIDATING_WORKFLOWS(3)**: Application is instantiating and validating workflows.
- **RECOVERING_WORKFLOWS(4)**: Application is recovering workflows.
- **STARTING_WORKFLOWS(5)**: Application is starting workflows.
- **FINALIZING(6)**: Application is doing any operation that might occur in the `APPLYING_CHANGES` state after `STARTING_WORKFLOWS` and moving to `IDLE` again.

Pulse Publishâ

Pulse's publish process can also be monitored with the following metrics.

Metric	Description	Tags	Type
`pulse.publish.e2e.success`	Measures end-to-end publish time that ended in success.	• Standard Pulse application tags	Timer
`pulse.publish.e2e.failure`	Measures end-to-end publish time that ended in failure.	• Standard Pulse application tags	Timer

Note that these timers also track the count of each occurrence.

What to monitorâ

These metrics may help identify bottlenecks in the publishing process.

JVM metricsâ□□

Each Pulse Server and Pulse Application instance live in their own JVM process. Therefore, JVM metrics for each of these processes are available.

How to measureâ□□

JVM metrics are available as `letmeknow.jvm`. Youâ□□ll find many different metrics under this namespace, from GC to Memory.

What to monitorâ□□

Look into at least GC and Memory metrics. These give you an idea of the healthiness of the process.

Apps Service DB Connection Poolâ□□

There are several DB connections used in Pulse. However, one of the most important to monitor is the one used by the Apps Service. This service runs in Pulse Server and is responsible for handling all metadata management.

How to measureâ□□

The metrics for all DB pools are available under the `pulse.dbPool` prefix. To capture a specific pool, use the `poolName` tag. In this case, for the Apps Service DB connection pool, look for `poolName=appsService`.

There are several metrics that can be captured:

- **Connections usage**: `pulse.dbPool.usage`
- **Connection errors and timeouts**: `pulse.dbPool.connectionErrors`
- **Connection creation throughput and latency**: `pulse.dbPool.connectionCreationTimeStats`

What to monitor in Apps Service DB Connection Poolâ□□

These metrics detect underlying issues with connecting to the DB. For example, if the time to create a new connection takes much more than the usual average, or if the pool usage is always at 100% capacity, then thereâ□□s likely an issue.

Data Transfer Pipeline Serviceâ□□

To support the Data Transfer Pipeline, there is a workflow service that forwards workflow messages and feedback updates to GCP.

How to monitorâ□□

There are three main aspects that can be monitored on the DTP workflow service:

- Batch executor
- Dump storage
- Processing metrics

The metrics have further breakdown using tags for the channel and processing type. Expect to see the following combinations:

- `channel=MESSAGE,processingType=NORMAL` for workflow events being processed for the first time
- `channel=FEEDBACK,processingType=NORMAL` for feedback updates being processed for the first time
- `channel=MESSAGE,processingType=RECOVERY` for workflow events being retried
- `channel=FEEDBACK,processingType=RECOVERY` for feedback updates being retried

Batch Executorâ□□

The following metrics expose the internal batch executors used by the workflow service:

Metric	Description	Tags	Type
`pulse.workflow.dtp.batch.executor.pool.activeThreadsCount`	Number of active threads.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.batch.executor.pool.maxPoolSize`	The maximum pool size.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.batch.executor.pool.threadsUsageRatio`	The current thread usage ratio.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.batch.executor.pool.taskQueueCount`	The current size of the task queue.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.batch.executor.pool.taskQueueMaximumSize`	The maximum size of the task queue.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.batch.executor.pool.taskQueueUsageRatio`	The current usage ratio of the task queue.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.batch.executor.service.completedFlushes`	The number of completed flushes.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.batch.executor.service.deadletterEvents`	The number of deadletter events.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.batch.executor.service.failedFlushes`	The number of failed flushes.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.batch.executor.service.lastFlush`	The timestamp of the latest flush.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.batch.executor.service.pendingQueueSize`	The size of the pending queue.	• `workflowId` • `channel`	Gauge

Metric	Description	Tags	Type
		• `processingType`	
`pulse.workflow.dtp.batch.executor.service.pendingQueueUsageRatio`	The usage ratio of the pending queue.	• `workflowId` • `channel` • `processingType`	Gauge

Dump Storageâ□□

When enabled, the Dump Storage will store failed messages and feedback updates in disk so they can be retried at a later time. Note that for these metrics, the processing type will always be `RECOVERY`. The following metrics are available:

Metric	Description	Tags	Type
`pulse.workflow.dtp.dump.storage.dump.entries.retry.counter`	The number of entries in disk that will be retried.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.dump.storage.dump.entries.failed.counter`	The number of entries in disk that have exhausted all retries.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.dump.storage.retry.queue.size`	Number of in-memory entries waiting to be written to the retry folder.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.dump.storage.failed.queue.size`	The number of in-memory entries waiting to be written to the failed folder.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.dump.storage.recovery.action.latency`	The latency of the recovery job.	• `workflowId` • `channel` • `processingType`	Gauge
`pulse.workflow.dtp.dump.storage.recovery.job`	Whether the auto recovery job is running or not.	• `workflowId` • `channel` • `processingType`	Boolean

Processing Metricsâ□□

These metrics provide a high level view of how many messages and feedback updates are being processed (successfully and otherwise) by the service:

Metric	Description	Tags	Type
`pulse.workflow.dtp.processing.numberOfReceivedMessages`	The number of messages received.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.processing.numberOfSuccessfullyProcessedMessages`	The number of successfully processed messages.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.processing.numberOfFailedRecoverableMessages`	The number of messages that failed to be processed but were sent for recovery.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.processing.numberOfFailedNonRecoverableMessages`	The number of messages that failed to be processed and are not recoverable.	• `workflowId` • `channel` • `processingType`	Counter
`pulse.workflow.dtp.processing.messageLatency`	The latency of the processing.	• `workflowId` • `channel` • `processingType`	Timer

What to monitor in Data Transfer Pipeline Serviceâ□□

It's important to keep track of spikes in messages that failed to be processed. If they are recoverable this may suggest connectivity issues with GCP but if they are not recoverable they may indicate data or integration issues.

It's equally important to keep track if failed messages are being successfully retried or piling up in disk.